

WebCAP: Web Data Collection Via Automated Program Synthesis

José Fragoso Santos

- INESC-ID, Instituto Superior Técnico, University of Lisbon
3rd meeting of the Statistical Scraping Interest Group
The Social Hub, Porto, September 2026



The team



Pedro Aldeia



Francisco Gouveia



Frederico Ramos



José Afonso



João Desidério

The project

- **Objective.** Automatic construction and deployment of web scrapers for the Portuguese public sector.
- **Funding.** FCT grant 2024.07393.IACDC, under the national programme on artificial intelligence, data science and cybersecurity for public administration.
- **Duration.** One year, at INESC-ID and Instituto Superior Técnico, University of Lisbon.
- **Case studies.** Agri-food retail prices, housing prices, and employment offers in Portugal.

Partners

- **GEE, Office for Strategy and Studies, Ministry of the Economy.** Founding partner. Defined the three case studies and the data requirements.
- **Statistics Portugal (INE), the national statistical institute.** Requirements for price and labour market collection, and the methodological constraints.
- **Other public bodies that have asked for the technology.** Banco de Portugal, the tax and customs authority, the competition authority, the public employment service, and municipalities.

This talk, in three parts

I. What we set out to build

An AI system that turns one sentence into a deployed web scraper.

Why a statistical institute would want one.

The architecture we proposed in 2025.

II. What actually happened

We built it and evaluated it on 388 candidate websites.

The two-stage idea holds up. The pipeline around it is too rigid to generalise.

Coding agents learned to write scrapers directly.

III. Where we are going now

Generation is no longer the hard problem. Trust is.

WebCAP 2.0: prove that a scraper does what was asked of it.

What we need to get there.

What we got wrong, what survived, and what we are building now.

Part I

The project we set out to build

Motivation, promise, and why it matters for official statistics.

High-quality data is fundamental for effective decision making



Businesses



Public Policy



AI Algorithms

But a lot of valuable data is scattered across the web in unstructured formats that are not easily accessible programmatically

How can we obtain actionable information from unstructured web data?



Software engineers use **web scrapers**:

Specialised programs designed to systematically parse and retrieve targeted information from websites

Extremely hard to create and deploy:

- Require **specialised labour**
- Time-consuming **reverse engineering**
- Significant **deployment challenges**

Our approach

AI powered development and deployment



WebCAP automatically generates:



A **scraper** for retrieving the targeted data



A **database** for storing the targeted data



A real-time **data tracker** for fetching the data



A **data API** for easily retrieving the targeted data

WebCAP applications



Housing



Retail



Banking



Why this matters to a statistical institute

What already happens today

Statistics Portugal runs web scrapers for the Consumer Price Index, monitoring retailers such as Zara, H&M and Pingo Doce.

Built in Python and MongoDB, one scraper per source.

Each site redesign breaks its scraper: roughly one manual repair per source per year.

Where the cost actually is

Writing the first scraper takes days.

Keeping a hundred of them alive, forever, takes a permanent team.

A broken scraper is rarely loud. It usually returns fewer rows, or the wrong ones.

The bottleneck is not writing scrapers. It is trusting them and keeping them alive.

Where the demand is

Collection targets identified in a workshop with two departments of Statistics Portugal: data collection and management, and methodology and information systems.

- **Travel and tourism prices for the CPI.** Airfares and hotel prices by destination and travel date. Not covered by automated collection today.
- **Online job advertisements.** Collection across platforms, deduplication across sources.
- **Short term accommodation.** Listings on Airbnb and Booking in the major cities.
- **Rental market aggregation.** Rental offers across real estate platforms, for near real time rental price dynamics.

Every one of these is a maintenance problem before it is a generation problem.

Part II

What we actually built

The insight, the architecture, the evaluation on 388 websites, and the results.

WebCAP architecture the prompt

“Generate a scraper that collects the prices of houses in Lisbon from the Idealista portal, going through every listing available”



Web scrapers: an example

```
<div class="item-info-container">
  <picture class="logo-branding">
    <a data-markup="listado:logo-agencia" href="/pro/jll-residential-portugal/" title="JLL Residential Portugal">
      
    </a>
  </picture>
  <a aria-level="2" class="item-link" href="/imovel/34282142/" role="heading"
    title="Apartamento T4 na Rua Padre António Vieira, 6, Bairro Azul - Parque Eduardo VII, Avenidas Novas">
    Apartamento T4 na Rua Padre António Vieira, 6, Bairro Azul - Parque Eduardo VII, Avenidas Novas
  </a>
  <div class="price-row">
    <span class="item-price h2-simulated">
      1.390.000
    </span>
    <span class="txt-big">
      €
    </span>
    <span>
    </span>
    <span class="pricedown">
      <span class="pricedown_price">
        1.490.000 €
      </span>
      <span class="pricedown_icon icon-pricedown">
        7%
      </span>
    </span>
  </div>
  <div class="item-detail-char">
    <span class="item-detail">
      T4
    </span>
    <span class="item-detail">
      257 m² área bruta
    </span>
    <span class="item-detail">
      1º andar com elevado
    </span>
  </div>
</div>
```

original price

price reduction

How to get the **prices** automatically?

Web scrapers: an example

```
def extract_xpath(page_html: str):
    def first(e, path): return next(iter(e.xpath(path)), None)

    def text(e, path):
        white_space = re.compile(r'\s+')
        if e is None:
            return None
        string = e.xpath(f"string({path})") if path else e.xpath("string()")
        cleaned = re.sub(white_space, ' ', string).strip()
        return cleaned or None

    tree = html.fromstring(page_html)
    listings = tree.xpath("//main[contains(@class, 'listing-items')]")
    data = []

    for l in listings:
        info = first(l, "//*[@class='item-info-container']")

        title = text(info, "//*[@class='item-link']")
        price = text(
            info, "//*[@class='price-row']//span[contains(@class, 'item-price')]")
        parking = text(
            info, "//*[@class='price-row']//span[contains(@class, 'item-parking')]")

        detail_spans = info.xpath(
            "//*[@class='item-detail-char']//span[contains(@class, 'item-detail')]")

        details = [text(span, '') for span in detail_spans]
        bedrooms, area, *_ = details

        data.append((title, bedrooms, area, parking, price))

    return data
```

Obtains the **required data** from the **HTML code** of the page

The insight

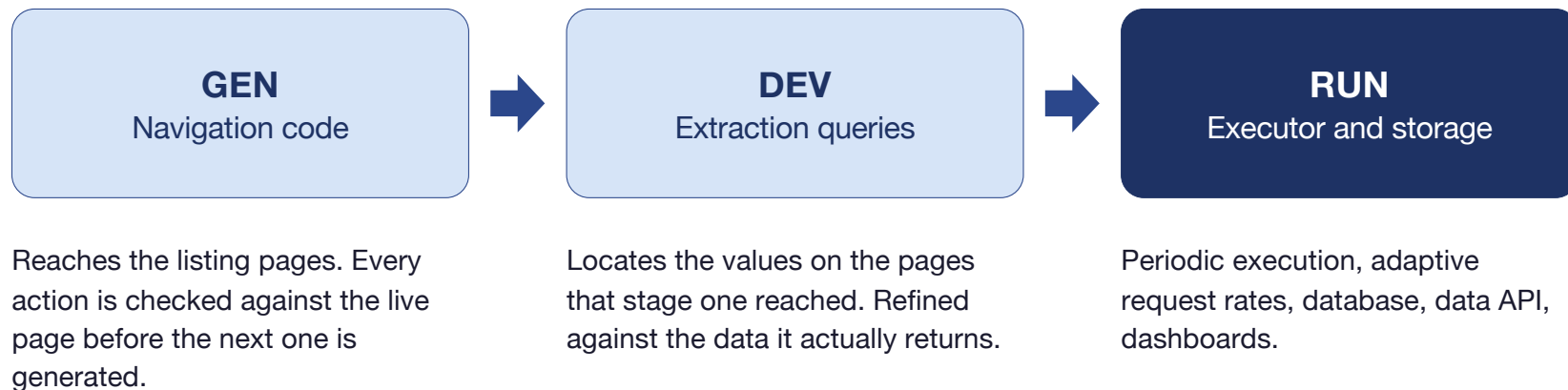
Do not ask a model for the scraper.

Ask for the navigation first, then the extraction, and mediate both through a tool you control.

- **Stage one synthesises the navigation code.** How to reach the pages that hold the data.
- **Stage two synthesises the extraction queries.** Which elements carry the values.
- **WebCAP is the vehicle for both.** It drives the browser and validates every step.

Navigation and extraction may fail for different reasons. Their synthesis and repair require different data.

Generation in two stages



In the preliminary WebCAP design the navigation code was written by hand. Moving it to automatic generation is where the difficulty turned out to live.

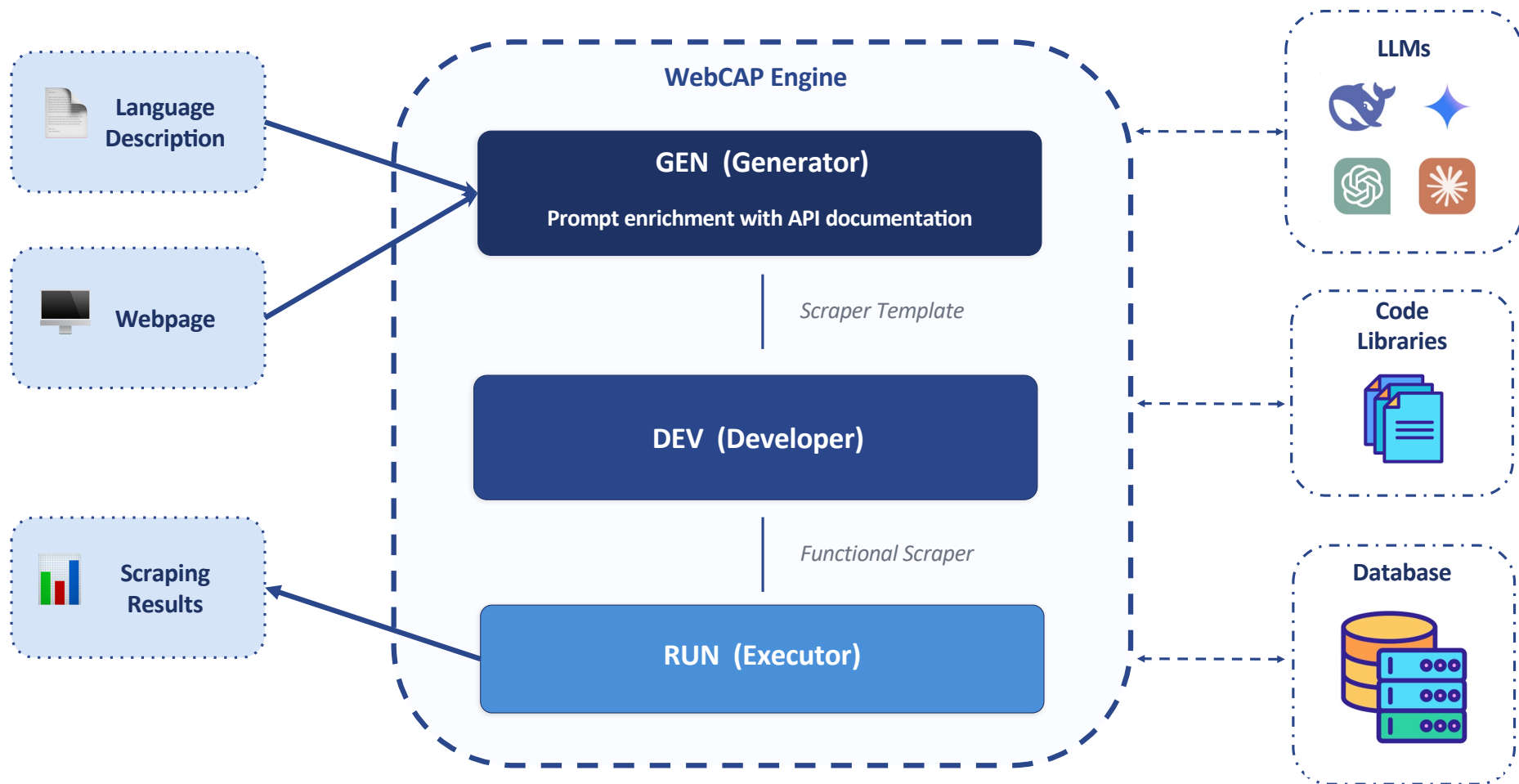
WebCAP design the scraper template

```
1 from webcap import webcaplib
2
3 def extract_prices(url):
4     prices = []
5     visit(url)
6
7     while(True):
8
9         prices += webcaplib.fetch("Preços das casas")
10        next_button = webcaplib.fetch("Botão seguinte")
11
12        if next_button is None:
13            break
14
15        click(next_button)
16    return prices
```

WebCAP design the generated scraper

```
1 from webcap import webcaplib
2
3 def extract_prices(url):
4     prices = []
5     visit(url)
6
7     while(True):
8         prices += webcaplib.fetch("//span[contains(@class,
9             ↪ 'item-price')]/text()[normalize-space()]")
10
11         next_button = webcaplib.fetch("//li[contains(@class, 'next')]")
12
13         if next_button is None:
14             break
```

WebCAP design the pipeline



WebCAP design the library

- **webcap.visit**(url | element) => Visit a page
- **webcap.click**(element) => Click on a link/button
- **webcap.fetch**(#id, description) => Extract data
- **webcap.scroll_down**(max_scrolls) => Scroll down
- **webcap.store**(data, db) => Save data to the database



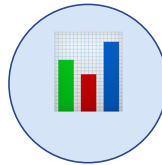
WebCAP design other functionalities

PDF Scraping



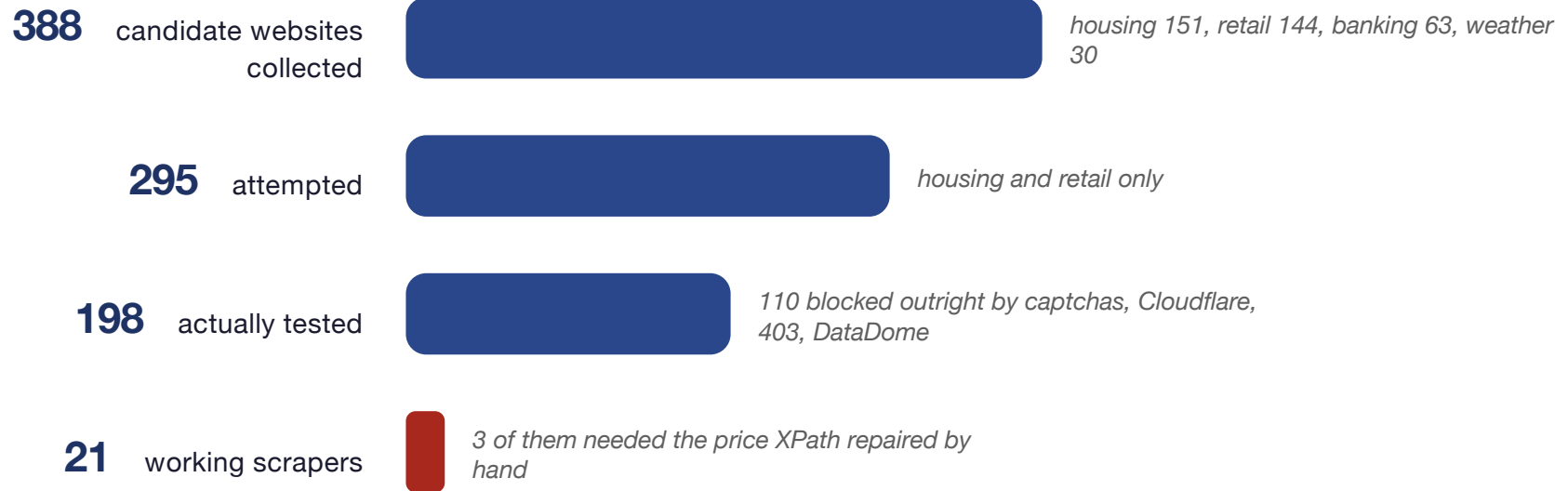
- Primitives for obtaining documents from the web and extracting their text
- Primitives for searching patterns in documents via automatic regular expression synthesis

Dashboard Integration (Grafana)



- Run the scraper continuously, store data in time-series databases
- Feed data directly into Grafana for real-time monitoring

How we evaluated it



The protocol

One hand written prompt per site, describing the navigation path in plain language.

What counted as success

The scraper runs end to end and returns the target values. Human in the loop allowed.

The results

198

sites tested
out of 388 collected

21

scrapers working
housing 14, retail 7

10.6%

success rate
with a human in the loop

- 3 of the 21 needed the price XPath repaired by hand. One of those also needed list indexing added to the generated code.

This is not a tuning problem. A pipeline that works on one site in ten is not the right pipeline.

Where it breaks

1. The search box is not a search box

Argenprop: what looks like a text field is a dropdown. Typing is not enough, you must pick an option.

Boliga: the query only takes effect after selecting the first suggestion.

2. You do not land where you asked to land

Centris: pressing Enter jumps straight to the first property, never to a listing page.

Several sites only show listings after a rent or buy radio button is chosen.

3. Pagination is not pagination

CapiFrance: listings advance through a horizontal carousel arrow, not a next page button.

Metro Peru: cookies and search work perfectly. There is simply no next page button.

4. The value is not one element

Comparis: the price is split across elements and comes back as a list, needing indexing.

athome.lu and **Ciencuadras:** the generated XPath had to be fixed by hand.

Every failure is a small, site specific mismatch. A rigid pipeline has nowhere to put that knowledge.

Meanwhile, the ground moved

*Second experiment: give a general purpose coding agent with web access the same task. No WebCAP, no library, no prompt engineering. Just “write me a scraper for this site (TypeScript + Playwright)”. Figures below are **GPT-5.6-Luna**.*

16 / 20

sites WebCAP solved

9 / 11

sites WebCAP failed on

6 / 10

sites never seen before

31 of 41 sites, 76 per cent.

WebCAP: 21 of 198, 11 per cent.

- We repeated the experiment with Claude Code and obtained comparable results.
- Sites that block outright (Costco, Jumbo) defeat both approaches equally.

What we got wrong

- **We optimised the stage that stopped being hard.** In 2025 the difficulty was generating code. It is not any more.
- **We mediated the model's access to the page.** Our API and our error messages became the ceiling on what it could understand.
- **The refinement loop had a fixed intermediate artefact.** A pre-determined format, with nowhere to put the knowledge each site specific failure needed.
- **Human in the loop was an escape hatch, not a design.**

The insight was right. The vehicle was wrong.

What survives

- **The dataset.** 388 sites, one prompt each, and ground truth about how each one actually behaves. It becomes a validation benchmark.
- **The deployment layer.** Scheduling, adaptive request rates, storage, data API, dashboards.
- **One open problem.** Models do not prove that the generated scraper is correct, and no existing tool does it for them.

The last one is the one worth building a project on.

Part III

WebCAP 2.0

Two decisions. Scrapers are now plain Playwright programs, written directly by a coding agent. We stop synthesising them, and build the layer that checks them.

The question has changed

The old question

Can a machine write this scraper?
Largely answered, and not by us.

The new question

How do we know the scraper it wrote is right, without
a person reading the code and watching the browser?

- **Wrong at birth.** The scraper runs, returns numbers, and the numbers are the wrong numbers: a neighbouring field, a filtered subset, page one of forty.
- **Wrong three months later.** The site is redesigned, the scraper keeps running, and the series quietly degrades until someone audits it.

For a statistical institute, a silently wrong scraper is worse than no scraper.

Idea 1 make the scraper say what it did

- Instrument the Playwright program so that every action it performs is recorded as it runs.
- A thin wrapper around the Playwright API. No new library, no new language.
- Works on any scraper, including one a coding agent wrote in plain Playwright.

```
visit("supercasa.pt")
click("#search-box")
fill("#search-box", "lisboa")
click("li.suggestion:first")
fetch(price) x 24
click("a.next")
fetch(price) x 24
click("a.next") ...
```

The execution trace: an ordered, machine readable record of what the scraper did to the page.

This is the part that costs almost nothing and that no current tool provides.

Idea 2 say what it should have done

A small specification language over scraper actions:

```
visit(url) ; click(searchBox) ; fill(searchBox, city) ; click(option) ; ( fetch(price)+  
; click(next) )*
```

- **Four operators.** Sequence, choice, repetition, optional. Regular expressions over actions.
- **Two ways to get one.** A person writes it, or it is synthesised from a textual prompt, which already says the same thing in prose.
- **It is small enough to read.** Five lines of specification can be checked in seconds. A hundred lines of Playwright cannot.

Human effort moves from reviewing code to approving intent.

Idea 3 check the trace against the specification

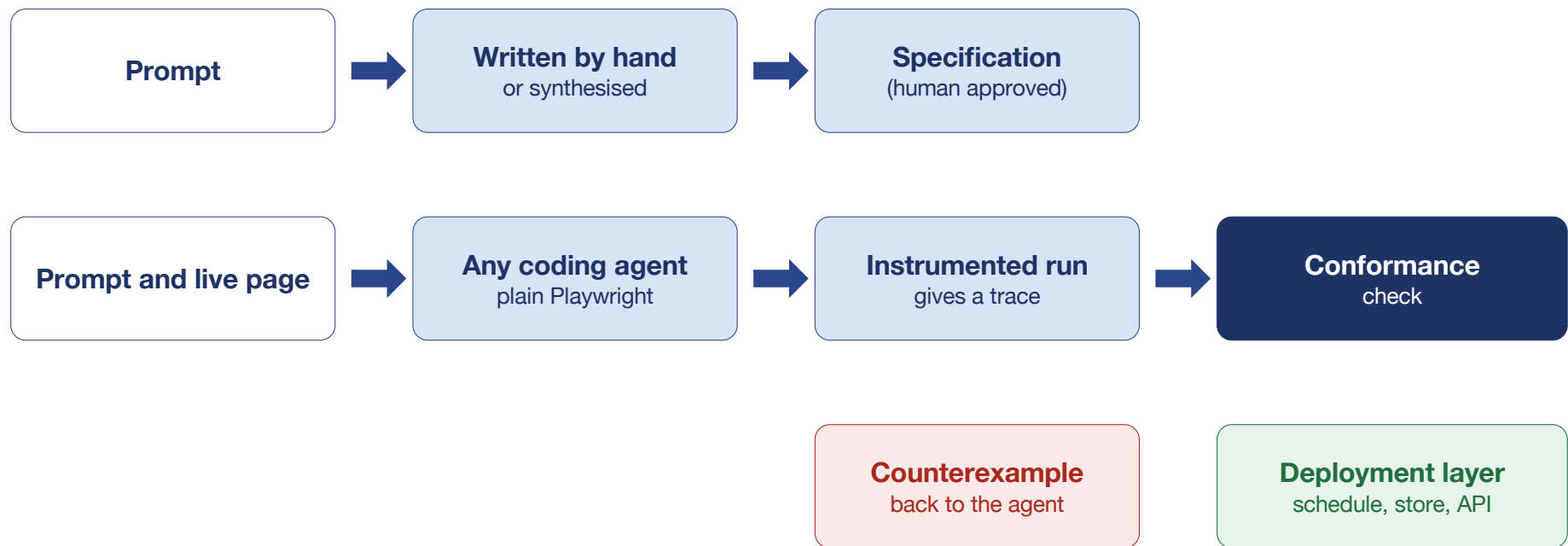
Does the recorded trace match the specification? A conformance check, in essence regular matching over actions.

When it fails, the mismatch is localised, and that is the point:

- **fill matched, click(option) did not.** The search box was a dropdown and nothing was selected. (Argenprop)
- **fetch(price) matched zero times after the first page.** Pagination silently stopped.
- **click(next) never matched.** There is no next page button, it is a carousel. (CapiFrance)
- **fetch(price) matched twice per listing.** The price is split across two elements. (Comparis)

A failed match is an actionable error message, which the WebCAP 1.0 refinement loop never had.

WebCAP 2.0



- Re-run the conformance check on every scheduled execution. The specification becomes a permanent monitor, not a one-off test.

What this buys

- **Termination on evidence.** The loop stops when the trace matches the specification, not when the model says it is done.
- **Continuous monitoring.** A site redesign shows up as a specification violation on the day it happens.
- **Human effort in the right place.** A person approves five lines of specification instead of reading a hundred lines of Playwright.
- **Agent agnostic.** It works with whichever coding agent is best this month, so we stop competing with model providers.

The contribution moves from synthesis to verification.

Open questions

- **How expressive must the specification language be?** Are action sequences enough, or do we need data constraints?
- **How good can automatic synthesis get?** Writing a spec is cheap, so this decides how much work is saved, not whether it works.
- **What counts as one action?** A single click can trigger five page events.
- **Can a counterexample be fed back automatically?** Or does a person have to read it?
- **What does correct mean statistically?** Conformance is not representativeness.

WebCAP 1.0 asked a machine to write scrapers. WebCAP 2.0 asks it to prove them.